Week2

- 理解 agent。**Agent = 大模型 + 工具 + 循环 + 目标+中止，相当于给 ai 装上了手脚和感官，能够自行决策，自行判断是否需要调用工具以及用什么工具来完成任务，同时能够知晓任务完成进度，如果没有完成任务那么就进行循环，直到出现可验证的完成证据，或触及步数/预算边界为止。**

"模型自己认为完成"恰恰是 Agent 最经典的失败模式，不是它跑完的条件：

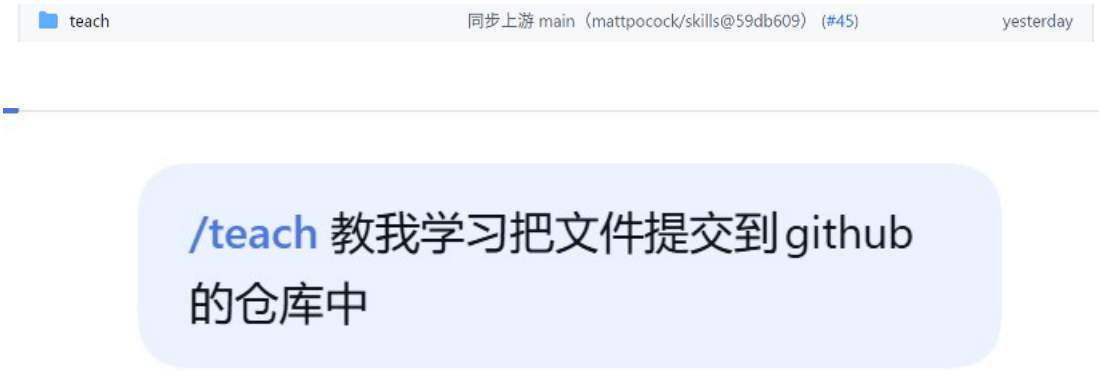
- 它说"已生成文件"，其实路径写错了，文件不存在
- 它说"测试都通过了"，其实命令报错，它把 stderr 忽略了
- 它说"已修复 bug"，其实只是改了注释
- 它陷入循环，反复说"快完成了"，因为每轮它都"认为"自己在推进

2.理解使用 skill。

skill 可包括提示词（必须），脚本，模板，数据等内容，使用/或使用对应提示词可以调用 skill，skill 相当于是对各种能力进行封装，需要时便可以调用。

Skill 应装在智能体运行架构（harness）能扫描到的根目录文件的下一级。

```
<root>/my-skill/
├── SKILL.md          # 必需
├── references/       # 可选：长文档、规范、示例
├── scripts/          # 可选：可执行脚本
└── assets/           # 可选：模板、素材、数据
```



3.linux

要跑通一个程序需要的步骤

确认环境 → 装依赖 → 摆好路径与权限 → 编译/解释执行 → 运行与守护 → 排错。

确定环境:

```
cat /etc/os-release          # 发行版与版本（决定用 apt 还是 dnf/yum）
uname -m                     # CPU 架构：x86_64 / aarch64
uname -r                     # 内核版本
ldd --version | head -1      # glibc 版本（预编译二进制的硬门槛）
echo $SHELL; whoami; id      # shell 与身份
nproc; free -h; df -h        # CPU/内存/磁盘，决定能不能编译、够不够放数据
```

根据对应的环境装好依赖。

依赖分为 1.系统层（如 c 语言编译器（将 c 语言编译成计算机能够理解的机械码）以及解压库等）2.库层（如开发包（程序编译是需要）和运行时库程序运行时需要））3.语言层（各种语言自己的包管理器）

路径和权限

权限	对**文件**	对**目录**
r 读	读取文件**内容**（`cat`）	列出目录里的**文件名**（`ls`）
w 写	修改文件**内容**（`echo >>`）	在目录里**创建、删除、重命名**条目
x 执行	把文件当**程序/脚本**运行	**进入**目录（`cd`）、穿过它访问里面的文件

```
-rwxr-xr--  1 app app 12345 Oct 29 10:00 start.sh
|└┘└┘└┘
| |  |  └── 其他人 (other) : r--  = 只读
| |  └───── 属组 (group) : r-x  = 读 + 执行
| └──────── 属主 (owner) : rwx  = 读 + 写 + 执行
└─────────── 类型: - 普通文件, d 目录, l 软链接
```

其中 rwx 可改为数字记法，r=4，w=2，x=1

路径分为绝对路径和相对路径（绝对路径从根目录开始，相对路径从当前目录开始）

运行指令+绝对路径可以在任何目录运行程序，运行指令+相对路径只能在特定的目录中运行程序。但是通过绝对路径启动程序时程序寻找自己的文件时会在**启动时的工作目录中寻找**，所以会出现找不到应用自己的文件的情况，因此需要写脚本来进行启动（先进到程序所在的目录再启动）

编译，解释

c/c++等需要编译器将源码编译成汇编**代码**再**汇编成**机器码（可执行文件），最后由链接器链接成可执行文件，计算机才能执行。

python 等语言需要解释器，运行时边读边解释。

.运行程序

,/程序名运行程序 ./程序名 &后台运行程序

排错

1. **看退出码**：`echo $?`，`0` 成功，非 0 是线索（`127` 命令找不到、`126` 无执行权限、`137` 被 SIGKILL 通常是 OOM、`139` 段错误）。
2. **看 stderr**：错误信息第一行往往就写了答案，别只看最后一行。
3. **看应用日志**：`journalctl -u myapp -n 200`、`tail -f app.log`。
4. **看系统层**：`dmesg -T | tail`（OOM、磁盘、驱动）、`df -h`（磁盘满）、`free -h`、`ulimit -n`（文件句柄）。
5. **看系统调用**：`strace -f -e trace=file ./myapp`，直接看它到底在找哪个路径、哪一步被拒绝。这是定位"路径/权限"问题最快的手段。
6. **看资源占用**：`lsof -p <pid>`、`ss -lntp`、`top -Hp <pid>`。

4.提交程序到 github 上

cd “目录路径”

git init 将当前目录生成.git 文件夹使目录成为【本地仓库+工作区】

git add .将该目录中的内容暂存到暂存区

git commit -m “***”**把暂存区的内容固化成一次版本快照，写进本地仓库（**.git

git remote add origin git@github.com:wanghouhua/**真实仓库名** 给本地仓库**登记一个远程地址**，并起个代号

git push -u origin main 把本地 main 分支**推送到远程** origin

1. cd C:\Users\王厚骅\Documents → 把当前终端的工作目录切到 Documents 文件夹，因为 git clone 会把仓库下载到"当前目录"下。
2. git clone git@github.com:wanghouhua/group-meeting-2026-fall.git → 把你 GitHub 上的 fork 完整下载到本地、新建同名文件夹，并自动把 origin 指向这个 fork。
3. cd group-meeting-2026-fall 进入刚克隆出来的仓库文件夹，之后所有 git 命令都在这个仓库里生效。
4. git remote add team git@github.com:HZCU-HPC-Team/group-meeting-2026-fall.git 额外登记一个名叫 team 的远程地址，指向团队原仓库，方便以后从它拉取最新代码。
5. git remote -v 列出已登记的所有远程地址和它们对应的 fetch/push 网址，用来确认 origin 和 team 都加对了。

6. git switch -c w02/whh 基于当前 main 的最新提交新建一个分支 w02/whh 并切过去，把作业改动隔离在这条分支上。

7. Copy-Item "...\week2\新建 文本文档.pdf" "W02\whh\week2-report-20261029.pdf" → 把桌面上的 PDF 复制进 W02/whh/ 目录，同时改名为符合仓库规范的英文小写文件名（原文件不动）。

8. git add W02/whh/ 把 W02/whh/ 下的新增或修改文件放进暂存区，标明"这些要进入下一次提交"。

9. git commit -m "report " → 把暂存区内容打包成一次版本快照，写进本地入库历史并附上说明文字 report

10. git push -u origin w02/whh → 把本地 w02/whh 分支推送到 origin（你的 fork）并在 GitHub 上建同名分支，-u 同时把它设为上游，以后直接 git push 即可（**这句还没执行**）

步骤	命令	这一句在做什么	位置
① 准备	—— 拿到一份「你能推」的副本		
0	（网页点 Fork）	复制一份可写副本	网页
1	cd C:\Users\王厚骅\Documents	切到放代码的父目录	本地终端
2	git clone git@github.com:wanghouhua/group-meeting-2026-fall.git	下载你的 fork，origin 指向它	网络→本地
3	cd group-meeting-2026-fall	进入仓库目录	本地终端
4	git remote add team git@github.com:HZCU-HPC-Team/group-meeting-2026-fall.git	登记团队仓库地址	本地配置
5	git remote -v	核对两个远程都正确	本地配置
② 隔离与改动	— 在分支上把作业摆好		
6	git switch -c w02/whh	切出作业分支，不污染 main	本地仓库
7	Copy-Item "...\week2\新建 文本文档.pdf" → "W02\whh\week2-report-20261029.pdf"	复制进目录并改合规名	本地工作区
8	git add W02/whh/	放进暂存区	本地暂存区
9	git status git diff --cached --stat	核对只提交自己的文件	本地暂存区
③ 提交	—— 把改动记成版本（本地）		
10	git commit -m "W02: add week2 report (whh)"	写成本地版本快照	本地仓库
④ 推送与 PR	— 此刻才真正上网		
11	git push -u origin w02/whh	推到 fork 并绑定上游	本地→fork
12	（网页）Compare & pull request → Create	请求合入团队 main	网页
⑤ 收尾	—— 合并后清理		
13	git switch main	切回 main	本地
14	git pull team main	拉回含你作业的最新 main	team→本地
15	git branch -d w02/whh	删已合并的本地分支	本地
16	git push origin --delete w02/whh	删 fork 上的同名分支	fork